

One Imperfect Page per Hundred: Testing Scribe on Data I Didn't Curate

I tested Scribe on 100 handwritten prescriptions with annotations supplied by someone else. I had not used these pages to tune the pipeline.

With the Qwen verifier, **97 pages were sent for human review**. Of the three accepted automatically, two were correct and one missed a medicine. With the Gemma verifier, 99 pages went to review and the single page accepted automatically was correct.

The review checks limited the number of unreviewed errors, but the pipeline still needed human help on almost every page. This post walks through what the run showed and what it did not establish.

Where this started

In [part 1](#), I built Scribe — a local pipeline that turns handwritten clinical forms into structured data and, more importantly, learned when *not* to trust a model's answer. The headline was that model confidence carried no signal, and disagreement between two independent reads did.

But every number in part 1 had a limitation.

I curated the gold answers. I wrote the extraction schema. I tuned the prompts against the same 23 pages I was scoring on. Even done carefully, that means the system and the test were developed together.

So I wanted a test set I had no hand in.

The external test

I found a [public dataset of 100 handwritten Indian prescriptions](#), where the ground truth — the list of medicines on each page — was written by the dataset's author, not by me.

The rules I set for myself:

- the extraction schema gets written once, from the task description, before seeing results
- the prompts stay frozen exactly as they were in part 1
- the author's answer conventions are the answer key, not mine

Two things make this set difficult. The handwriting is real doctors' cursive — not the careful volunteer writing my benchmark was built from. And the drug names are Indian brand names the models have seen little of.

That difficulty is useful: a system built for clinics in one country will eventually meet forms from another.

Reading accuracy dropped sharply

Brand names came back right 38% of the time.

CEPODEM became **Capelin**. *ESOTAB* became **Esotrib**. *NIVEOLI* became **Ulfah Niveoli**.

These are the same kind of confident misreads that affected patient names in part 1, and they happened more often here: the handwriting is harder and the vocabulary is unfamiliar. (The two evaluations use different measures — field-level accuracy on the development set, brand-name recall here — so I’m not putting a before-and-after number on it.)

I ran both of my best configurations, which differ only in which model does the second read. They produced **identical reading results** — same primary model, so the underlying reading was unchanged; the verifier only affected how much got flagged.

That repeats part 1’s central lesson: **verification cannot fix reading. It can only catch it.**

No prompt I own fixes the reading either. For the local vision models I tested, hard cursive remains a major limitation — and a [recent benchmark on South African maternity records](#) reported similar difficulty even with frontier cloud models.

The review gate held

This is the part that mattered to me.

On these harder pages, the models were much less willing to claim certainty: most confidence scores fell to around 0.60–0.70, and **97–99% of pages were sent for human review**.

For this input, “a person must look at this” *is* the correct output.

Auditing the escapes

With the Qwen verifier, three pages out of a hundred skipped human review. I checked each one by hand.

- Two were correct — single-medicine prescriptions where the extracted medicine matched the annotation.
- One was an eleven-medicine page that got ten of them right and **dropped one** (**BETNESOL INJ**).

So the imperfect page is one out of 100 pages tested — but also one out of only three pages accepted without review. Those two denominators tell different stories. The result shows the

pipeline routed almost everything to a person; it does not demonstrate high accuracy among automatically accepted pages, because there were too few of them to say.

With the Gemma verifier, one page was accepted automatically, and it was correct.

Part 1 ended with a rule: *count what escapes, and you're grading the system*. This is what that looks like in practice — a specific, audited count of unreviewed errors rather than a single accuracy number.

The test also caught a bug in my scorer

One more thing this exercise caught — in my own tooling.

My first scoring pass penalized extracted rows for containing dose schedules like (1-0-1), even when they named the right drug. The scorer's precision check compared token sets in the wrong direction. External data debugs your evaluator, not just your model.

What I'm taking away

- **Reading accuracy didn't transfer to new handwriting; the review-gate behavior did.** The parts that held up were the system parts — confidence behavior, dual reads, the review queue. The reading itself did not.
- **Neither evaluation establishes performance on the intended target forms.** Part 1's benchmark is public data; this set is a different form type from a different country. The private held-out evaluation is what will measure the 98% target.
- **A high flag rate is not failure here.** Flagging 97% of pages the models genuinely couldn't read is correct behavior. The bad outcome would have been a normal flag rate with wrong drug names passing through unreviewed.

What's next

The held-out evaluation: real target-form registers, hand-filled, photographed, never published, created after every model's training cutoff. Those are the numbers that will actually count, and they'll be part 3 of this series.

And the checkbox problem from part 1 still needs its own detection stage, likely using the one model in my tests that produced no hallucinated values.

► **The numbers, for engineers (click to expand)**

This is part 2 of a series. Part 1: [Confidence Is Theater: What I Learned Building Scribe](#).

Code and methodology: github.com/kod201/scribe · My own benchmark gold set: [sohakanu/scribe-flow-gold](https://github.com/sohakanu/scribe-flow-gold)

All extraction ran locally on one machine; no page left it.